

**Data Mining & Business Intelligence (IT647)**

**Selected Topics In Data Science (IT688)**

**Information Technology Department**

**College Of Computer**

**Qassim University**

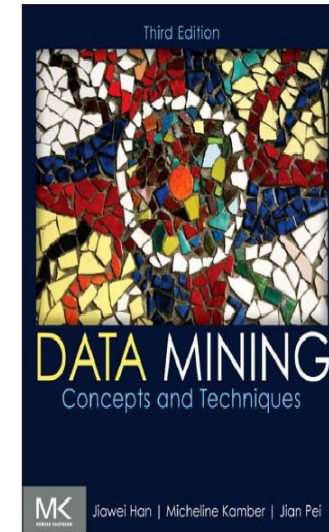
**DR.MONA ALSALEEM**

# Data Mining & Business Intelligence Course\_IT674

## Data Mining in BI

— Chapter 6 —  
Mining Frequent Patterns, Associations  
and Correlations

Text Book: “Data Mining: Concepts and Techniques”, Third  
Edition



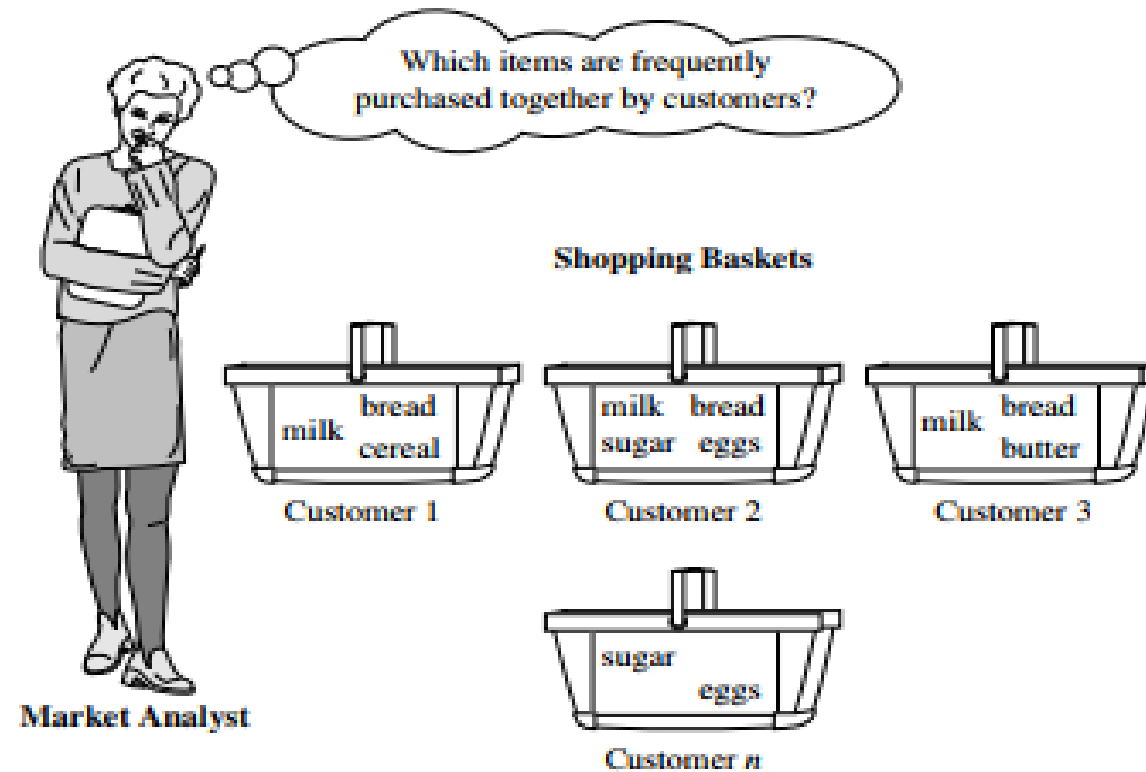
# Outlines

---

- Motivating Example
- Basic Concepts
- Frequent Itemset Mining Methods
- Application: Apriori Algorithm
- Summary

# Market Basket Analysis:

## A Motivating Example



### Market Basket Analysis

## Another example



- If customers who purchase computers also tend to buy antivirus software at the same time
- This can be described in association rule :  
**computer  $\Rightarrow$  antivirus software [support=2%,confidence=60%].**
- Typically, association rules are considered interesting if they satisfy both a minimum support threshold and a minimum confidence threshold.

---

## **Basic Concepts**

# Mining Frequent Itemsets

## ■ Itemset

- A collection of one or more items
  - Example:  $\{i_1, i_2, i_3\}$
- **k-itemset**
  - An itemset that contains **k** items

## ■ Support ( $\sigma$ )

- **Count:** Frequency of occurrence of an itemset
- E.g.  $\sigma(\{i_1, i_2, i_3\}) = 2$
- **Fraction:** Fraction of transactions that contain an itemset
- E.g.  $s(\{i_1, i_2, i_3\}) = 40\%$

## ■ Frequent Itemset

- An itemset whose support is greater than or equal to a **minsup** threshold,  $s(I) \geq \text{minsup}$

## ■ Problem Definition

- **Input:** A set of transactions **T**, over a set of items **I**, **minsup** value
- **Output:** All itemsets with items in **I** having  $s(I) \geq \text{minsup}$

| <i>TID</i> | <i>Items</i>             |
|------------|--------------------------|
| 1          | $\{i_1, i_2\}$           |
| 2          | $\{i_1, i_3, i_4, i_5\}$ |
| 3          | $\{i_2, i_3, i_4, i_6\}$ |
| 4          | $\{i_1, i_2, i_3, i_4\}$ |
| 5          | $\{i_1, i_2, i_3, i_6\}$ |

---

# **Frequent Itemset Mining Methods**

# 1- Finding Frequent Itemsets by Confined Candidate Generation

---

## Apriori Algorithm

- Apriori alg is proposed by R. Agrawal and R. Srikant in 1994.
- Its objective to Find Frequent Itemsets by Confined Candidate Generation.
- Created for mining frequent itemsets for Boolean association rules.
- To improve the efficiency of the level-wise generation of frequent itemsets, an important property called the *Apriori property* is used to reduce the search space.

“Any subset of a frequent itemset is also frequent”

- It has at least the support of its superset

# 1- Finding Frequent Itemsets by Confined Candidate Generation

---

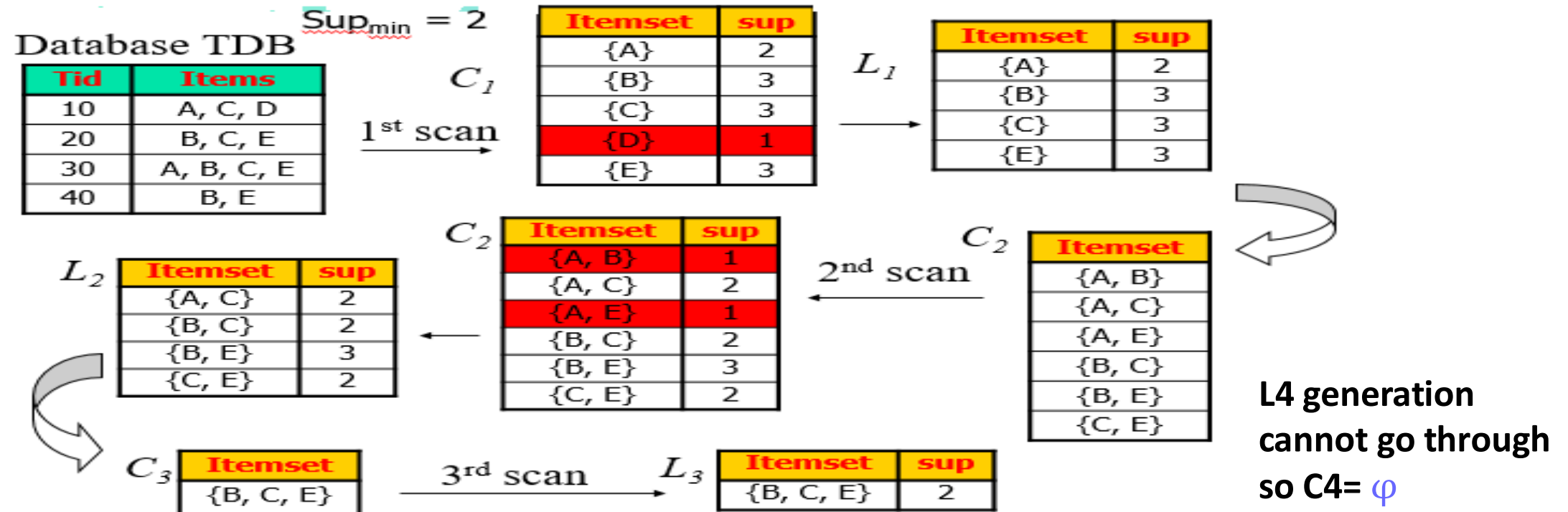
## Pseudo code

- **Join Step:**  $C_k$  is generated by joining  $L_{k-1}$  with itself
- **Prune Step:** Any  $(k-1)$ -itemset that is not frequent cannot be a subset of a frequent  $k$ -itemset
- **Pseudo-code:**
  - $C_k$ : Candidate itemset of size  $k$
  - $L_k$ : frequent itemset of size  $k$

```
 $L_1 = \{\text{frequent items}\};$   
for ( $k = 1$ ;  $L_k \neq C$ ;  $k++$ ) do begin  
     $C_{k+1}$  = candidates generated from  $L_k$ ;  
    for each transaction  $t$  in database do  
        increment the count of all candidates in  $C_{k+1}$  that are contained in  $t$   
     $L_{k+1}$  = candidates in  $C_{k+1}$  with min_support  
    end  
return  $C_k L_k$ ;
```

# 1- Finding Frequent Itemsets by Confined Candidate Generation

## Example



# Generating association rule

---

| Rule               | Support      |
|--------------------|--------------|
| $A \rightarrow C$  | $2/4 = 50\%$ |
| $B \rightarrow E$  | $3/4 = 75\%$ |
| $E \rightarrow B$  | $3/4 = 75\%$ |
| $BC \rightarrow E$ | $2/4 = 50\%$ |
| $CE \rightarrow B$ | $2/4 = 50\%$ |

## 2- Improving the Efficiency of Apriori

---

“How can we further improve the efficiency of Apriori-based mining?”

- Many variations of the Apriori algorithm have been proposed

### 1. Hash-based technique

- A hash-based technique can be used to reduce the size of the candidate  $k$ -itemsets,  $C_k$ , for  $k > 1$ .

## 2- Improving the Efficiency of Apriori

---

“

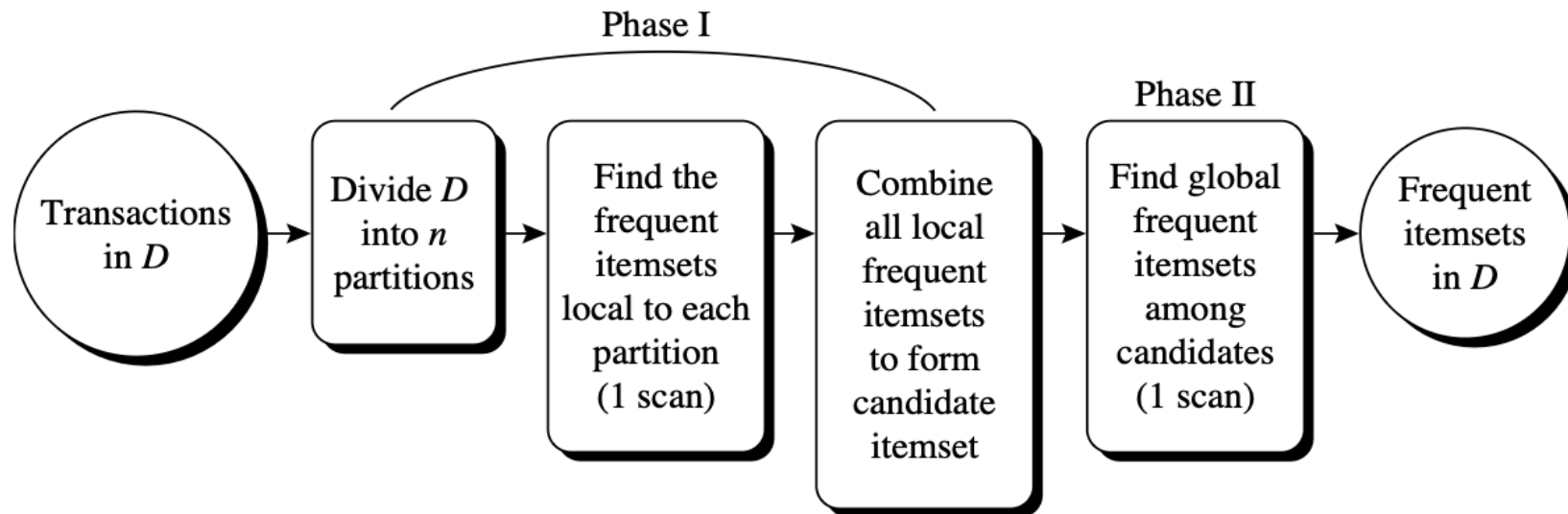
### 2. Transaction reduction

- Reducing the number of transactions scanned in future iterations,
- A transaction that does not contain any frequent  $k$ -itemsets cannot contain any frequent  $(k+1)$ - itemsets,

## 2- Improving the Efficiency of Apriori

### 3. Partitioning

A partitioning technique can be used that requires just two database scans to mine the frequent itemset. It consists of two phases.



## 2- Improving the Efficiency of Apriori

---

### 4. Sampling

- Sampling (mining on a subset of the given data)
- Its basic idea is to pick a random sample  $S$  of the given data  $D$ , and then search for frequent itemsets in  $S$  instead of  $D$ .
- The  $S$  sample size is such that the search for frequent itemsets in  $S$  can be done in main memory, and so only one scan of the transactions in  $S$  is required overall.

---

## **Application: Apriori Algorithm**

# Application

---

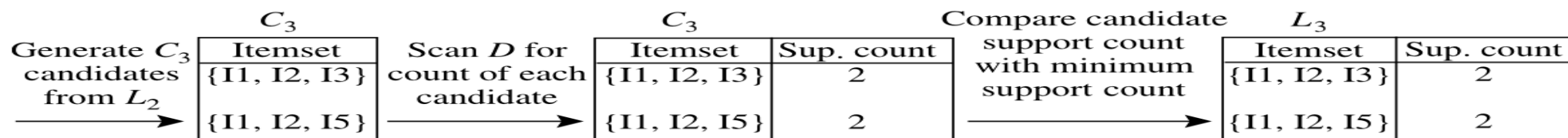
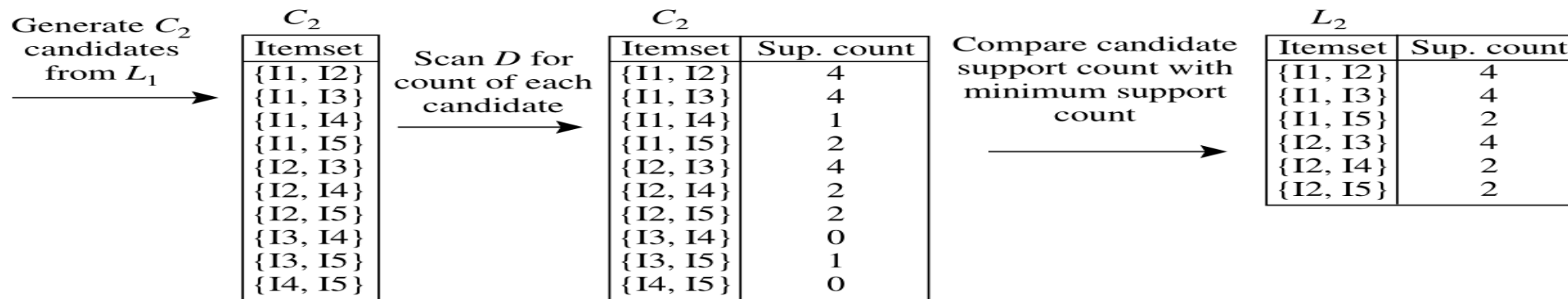
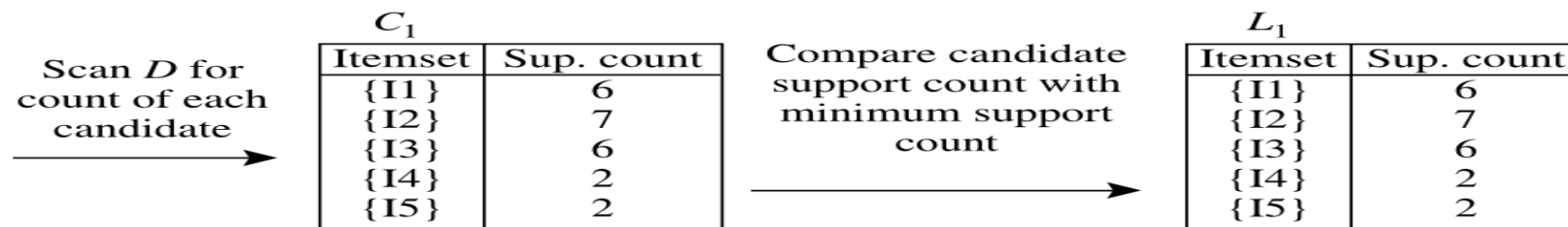
DataBase D

| TID  | List of Items  |
|------|----------------|
| T100 | I1, I2, I5     |
| T200 | I2, I4         |
| T300 | I2, I3         |
| T400 | I1, I2, I4     |
| T500 | I1, I3         |
| T600 | I2, I3         |
| T700 | I1, I3         |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3     |

- Consider a database, D, consisting of 9 transactions.
- Suppose min. support count required is 2 (i.e.  $\text{min\_sup} = 2/9 = 22\%$ )
- We have to first find out the frequent itemset using Apriori algorithm.

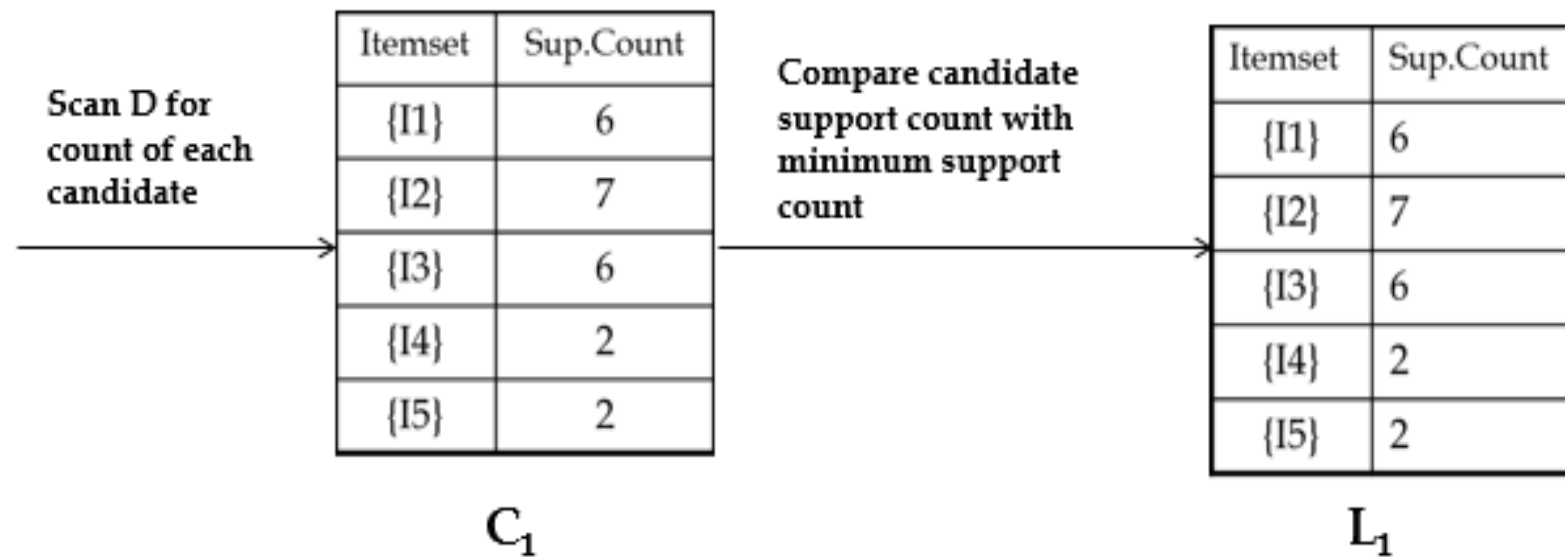
| TID  | List of Items  |
|------|----------------|
| T100 | I1, I2, I5     |
| T200 | I2, I4         |
| T300 | I2, I3         |
| T400 | I1, I2, I4     |
| T500 | I1, I3         |
| T600 | I2, I3         |
| T700 | I1, I3         |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3     |

# Step 1: Generating 1-itemset Frequent Pattern



| TID  | List of Items  |
|------|----------------|
| T100 | I1, I2, I5     |
| T200 | I2, I4         |
| T300 | I2, I3         |
| T400 | I1, I2, I4     |
| T500 | I1, I3         |
| T600 | I2, I3         |
| T700 | I1, I3         |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3     |

## Step 1: Generating 1-itemset Frequent Pattern



- In the first iteration of the algorithm, each item is a member of the set of candidate.
- The set of frequent 1-itemsets,  $L_1$ , consists of the candidate 1-itemsets satisfying minimum support.

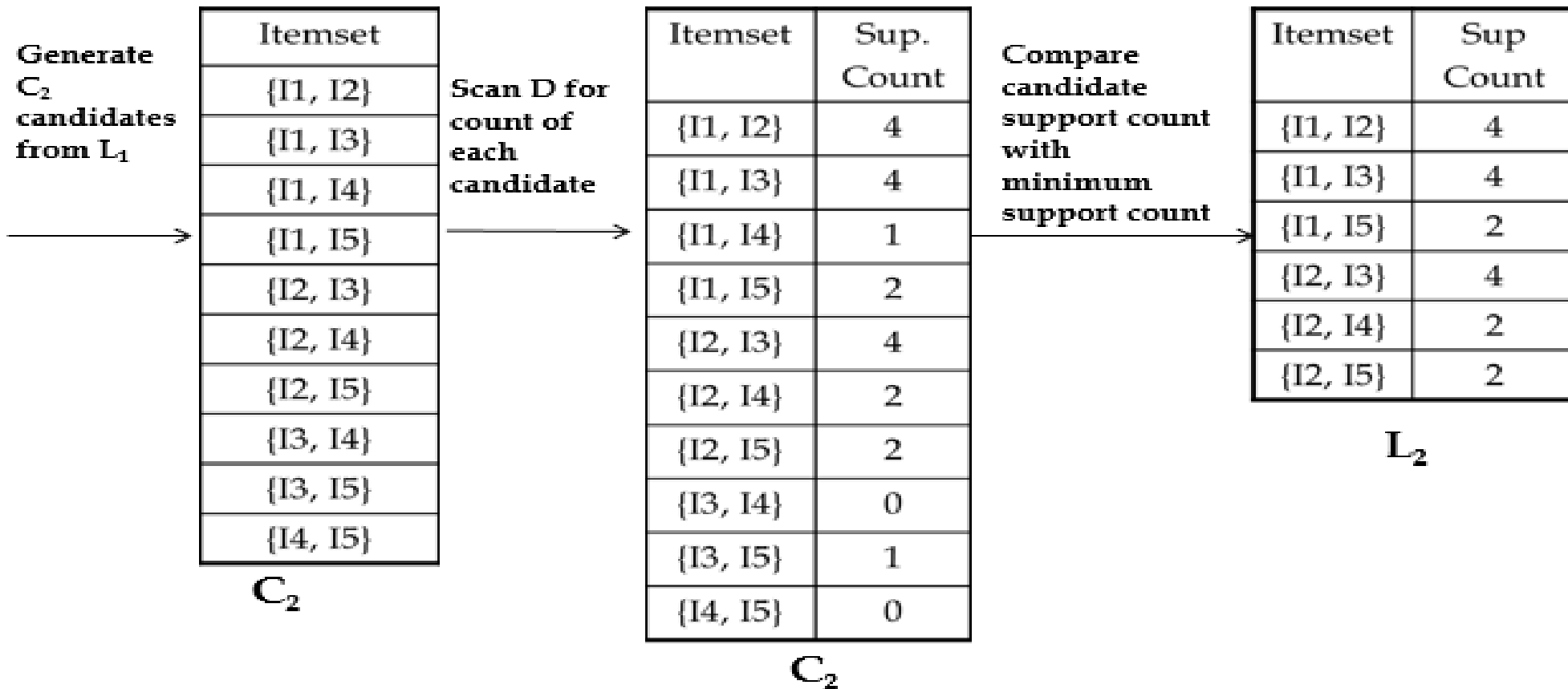
## Step 1: Generating 1-itemset Frequent Pattern

---

- To discover the set of frequent 2-itemsets,  $L_2$ , the algorithm uses  $L_1 \text{Join } L_1$  to generate a candidate set of 2-itemsets,  $C_2$ .
- Next, the transactions in  $D$  are scanned and the support count for each candidate itemset in  $C_2$  is accumulated (as shown in the middle table).
- The <sup>K</sup>set of frequent 2-itemsets,  $L_2$ , is then determined, consisting of those candidate 2-itemsets in  $C_2$  having minimum support.
- **Note:** We haven't used Apriori Property yet.

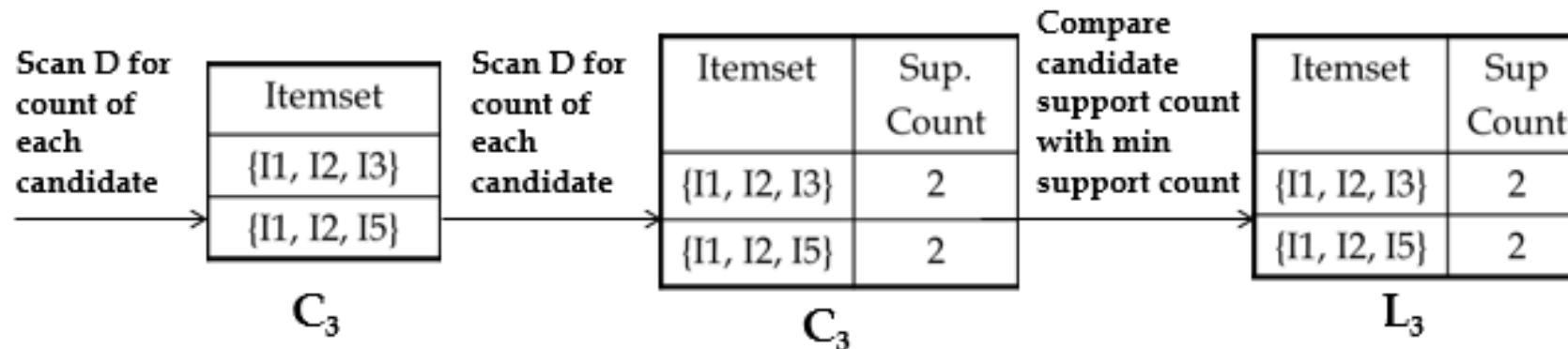
| TID  | List of Items  |
|------|----------------|
| T100 | I1, I2, I5     |
| T200 | I2, I4         |
| T300 | I2, I3         |
| T400 | I1, I2, I4     |
| T500 | I1, I3         |
| T600 | I2, I3         |
| T700 | I1, I3         |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3     |

## Step 2: Generating 2-itemset Frequent Pattern



## Step 3: Generating 3-itemset Frequent Pattern

- The generation of the set of candidate 3-itemsets,  $C_3$ , involves **use of the Apriori Property**.
- In order to find  $C_3$ , we compute  $L_2 \text{ Join } L_2$ .
- $C_3 = L_2 \text{ Join } L_2 = \{\{I1, I2, I3\}, \{I1, I2, I5\}, \{I1, I3, I5\}, \{I2, I3, I4\}, \{I2, I3, I5\}, \{I2, I4, I5\}\}$ .
- Now, **Join step** is complete and **Prune step** will be used to reduce the size of  $C_3$ . **Prune step helps to avoid heavy computation due to large  $C_k$ .**



## Step 3: Generating 3-itemset Frequent Pattern

---

Based on the **Apriori property** that all subsets of a frequent itemset must also be frequent, we can determine that four latter candidates cannot possibly be frequent. How ?

For example , lets take **{I1, I2, I3}**. The 2-item subsets of it are {I1, I2}, {I1, I3} & {I2, I3}. Since all 2-item subsets of {I1, I2, I3} are members of  $L_2$ , We will keep {I1, I2, I3} in  $C_3$ .

Lets take another example of **{I2, I3, I5}** which shows how the pruning is performed. The 2-item subsets are {I2, I3}, {I2, I5} & {I3, I5}.

BUT, {I3, I5} is not a member of  $L_2$  and hence it is not frequent **violating Apriori Property**. Thus We will have to remove **{I2, I3, I5}** from  $C_3$ .

Therefore,  $C_3 = \{\{I1, I2, I3\}, \{I1, I2, I5\}\}$  after checking for all members of **result of Join operation** for **Pruning**.

Now, the transactions in D are scanned in order to determine  $L_3$ , consisting of those candidates 3-itemsets in  $C_3$  having minimum support.

## Step 3: Generating 3-itemset Frequent Pattern

---

The algorithm uses  $L_3 \text{Join } L_3$  to generate a candidate set of 4-itemsets,  $C_4$ .

Although the join results in  $\{\{I1, I2, I3, I5\}\}$ , this itemset is pruned since its subset  $\{\{I2, I3, I5\}\}$  is not frequent.

Thus,  $C_4 = \varnothing$ , and algorithm terminates, having found all of the frequent items.

- Try To Generate The Association Rule From The Resulted Frequent Pattern !

# Summary

---

- Apriori is a fundamental algorithm for mining frequent itemsets and discovering association rules from transactional data.
- By exploiting the Apriori property, the algorithm efficiently reduces the search space through systematic join and prune steps.
- This enables businesses to uncover meaningful purchasing patterns, such as items frequently bought together.
- Despite requiring multiple database scans, Apriori remains widely used due to its simplicity, interpretability, and effectiveness in market basket analysis. Its insights support better decision-making in recommendation systems, cross-selling strategies, and retail optimization.

Thank you

